



Rhoda — Virtual Lab Assistant

Technical Implementation Report

Track 2: Virtual Lab Assistant (Conversational / Voice)

Hackathon: Rohde & Schwarz AI-Assisted Onboarding in Electrical Engineering

Institution: Kwame Nkrumah University of Science and Technology (KNUST)

Date: June 2026

Team: Afari (Stream I/O) · Gregory (RAG) · William (Instrument API) · Prince (Vision/Logging)

Repository: github.com/PhidLarkson/rohde-schwarz-assistant

Branch: xr-agent-and-logging

Table of Contents

1. Executive Summary
2. Project Metrics
3. System Architecture
4. Frontend Implementation
 1. XR Avatar System (rhode_schwarz.ts)
 2. Voice Pipeline (genai.ts)
 3. Hand Ray & XR Interaction (handRay.ts)
 4. Dashboard System (dashboard.ts)
 5. Text Chat Interface (chat.ts)
 6. RAG Retrieval Layer (rag.ts)
 7. Instrument Mock & Bridge (instrument.ts)
 8. Anomaly Detection (anomaly.ts)
 9. Progress Tracking (progress.ts)
 10. Vision / Component ID (vision.ts)
 11. Guided Workflows (workflows.ts)
 12. Session Logging (session.ts)
5. Backend Implementation
 1. Flask API Server (app.py)
 2. Instrument Control — Mock Backend
 3. Instrument Control — RTB24 Hardware Backend
 4. Confirmation Gate
 5. RAG with Sentence Transformers
6. SCPI Command Reference
7. API Endpoint Specification
8. Data Contracts
9. Demo Workflows
10. Testing
11. Completion Assessment
12. Oscilloscope Testing Procedure
13. Safety Considerations

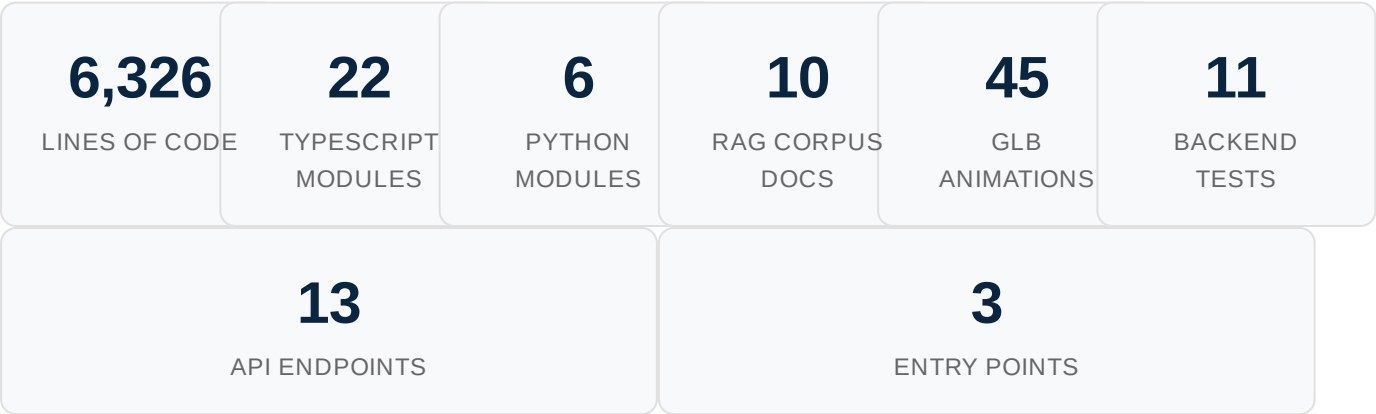
1. Executive Summary

Rhoda is a conversational AI virtual lab assistant designed to guide KNUST electrical engineering students through Rohde & Schwarz oscilloscope procedures. The system combines an XR avatar with voice interaction, a text-based chat interface, retrieval-augmented generation (RAG), real-time instrument control with safety-gated confirmation, anomaly detection for trace diagnosis, and adaptive progress tracking — delivering a complete Track 2 solution for the R&S AI-Assisted Onboarding hackathon.

The implementation spans **6,326 lines of source code** across 22 TypeScript modules and 6 Python modules, with a 10-document RAG corpus, 45 GLB animation assets, and 11 automated backend tests. The system supports both simulated and real R&S RTB24 oscilloscope hardware via USB or LAN connections.

Key deliverables met: Two guided workflows (simple + safety-critical), voice and text chat interfaces, safety-gated instrument configuration with explicit confirmation, conversation transcript export, instructor dashboard with failure point analytics, and comprehensive documentation for connecting to instruments or simulator.

2. Project Metrics

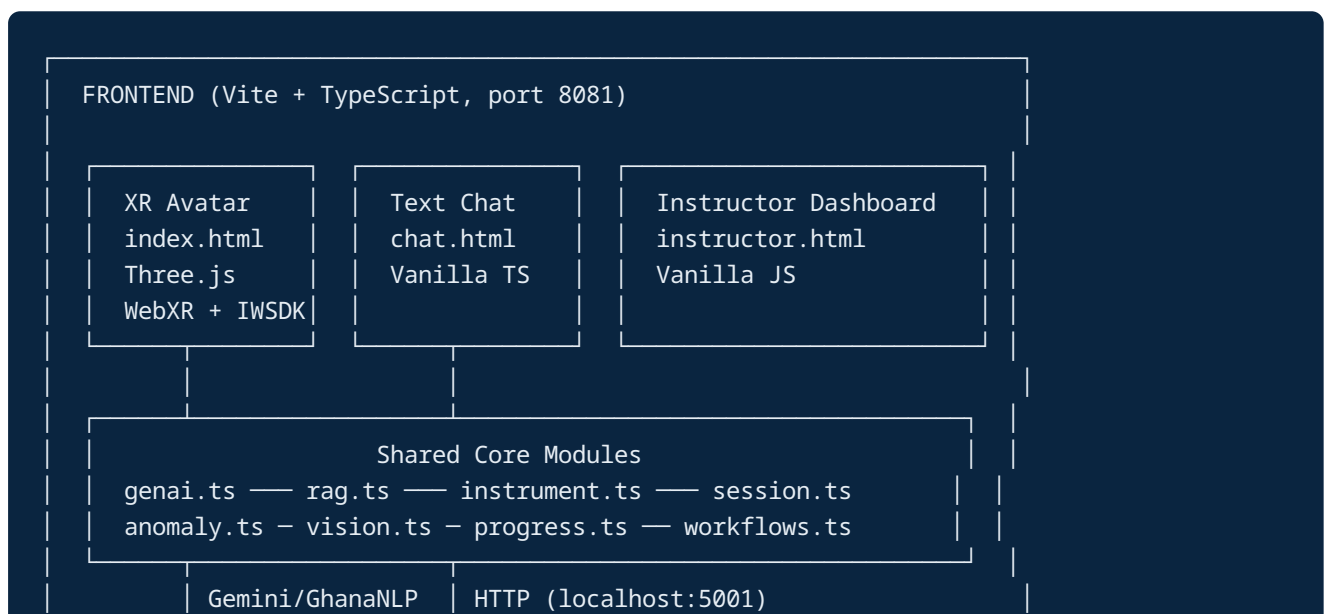


Module	File	Lines	Purpose
XR Avatar	rhode_schwarz.ts	1,405	Avatar, voice pipeline, lip sync, animations
Instrument Mock	instrument.ts	419	Client-side oscilloscope simulator + Flask bridge
Voice/AI	genai.ts	393	Gemini, GhanaNLP STT/TTS, translation
RAG (client)	rag.ts	363	Embedded keyword retrieval

XR Scene	index.ts	358	World setup, input wiring, placement
Instrument (server)	backends.py	307	Mock + RTB24 SCPI backends
Chat UI	chat.ts	304	Text chat entry point
Audio Recorder	audioRecorder.ts	296	Mic capture with WebAudio
Flask API	app.py	252	Unified backend server
Vision	vision.ts	249	Camera capture, component ID, live stream
Anomaly	anomaly.ts	237	Fault pattern matching + Gemini fallback
Scope Client	scope_client.py	214	ScopeClient with confirmation gating
Progress	progress.ts	201	Competency tracking, curriculum ordering
Hand Ray	handRay.ts	196	XR pointer beams and hit detection
Dashboard	dashboard.ts	195	XR panel UI system
Workflows	workflows.ts	189	Guided step-by-step procedures
Tests	test_api.py	174	11 integration tests
Interactions	interactions.ts	170	Interaction registry
Session	session.ts	168	Append-only event log

3. System Architecture

3.1 High-Level Architecture





3.2 Data Flow — Voice Conversation

```

User speaks —> AudioRecorder (WebAudio API, opus/webm)
                —> Gemini STT (base64 audio → text)
                —> Confirmation check (pending WRITE?)
                —> Anomaly keyword check
                —> Workflow step check
                —> RAG retrieval (keyword match)
                —> Instrument state snapshot
                —> Gemini LLM (multi-turn context + RAG + state + workflow)
                —> Instrument action detection (regex)
                —> Confirmation gate (if WRITE)
                —> Gemini TTS (text → WAV)
                —> Web Audio API playback + lip sync (AnalyserNode)
                —> Session log (localStorage + server POST)
  
```

3.3 Data Flow — Text Chat

```

User types —> chat.ts handleSend()
                —> Confirmation check / Workflow step check
                —> Anomaly detection (keyword trigger)
                —> RAG context retrieval
                —> askGemini() with history + RAG + instrument state + workflow context
                —> Instrument action detection + confirmation gate
  
```

- Display response + update instrument panel + progress bars
- Session log

3.4 Technology Stack

Layer	Technology	Version	Purpose
3D Rendering	Three.js (super-three)	0.177	WebGL scene, meshes, materials
XR Framework	Meta IWSDK	0.2.1	WebXR interactions, hand tracking, panels
Bundler	Vite	7.1	Dev server, HMR, multi-entry build
Language	TypeScript	5.5	Type-safe frontend code
LLM	Google Gemini 2.5 Flash	—	Reasoning, generation, vision, STT
TTS	Gemini TTS Preview	—	English speech synthesis (Vindematrix voice)
Twl NLP	GhanaNLP API	v1	Twl ASR, TTS, translation
Backend	Flask	3.0+	REST API server
RAG Embeddings	sentence-transformers	—	all-MiniLM-L6-v2 embeddings
Instrument	RsInstrument	1.124+	R&S VISA/SCPI driver

4. Frontend Implementation

4.1 XR Avatar System — `rhode_schwarz.ts` (1,405 lines)

The central module. The `RhodeSchwarzAssistant` class manages a GLB-based humanoid avatar with:

- **Animation system:** AnimationMixer with idle (9 variants), talking (6 variants), locomotion (18 variants), and expression categories. Lazy-loaded: primary idle loads first to unblock startup, remaining animations load in the background.
- **Lip sync:** Real-time AnalyserNode reads audio waveform data during TTS playback. Amplitude is normalized and mapped to morph target influences (jawOpen/mouthOpen) on the head mesh. Square-root curve prevents over-opening.
- **Blinking:** Periodic eye mesh visibility toggle (4.5–6s period, 150ms blink).
- **Head tracking:** Procedural lookAt with smooth slerp interpolation. Random glances in READY state; always faces user when LISTENING/PROCESSING/SPEAKING. Yaw/pitch clamped to $\pm 60^\circ$.

- **State machine:** READY → LISTENING → PROCESSING → SPEAKING → READY. Silence detection (configurable threshold, 1200ms / 600ms fast mode) auto-stops recording.
- **Multi-turn context:** 20-turn `ChatTurn[]` history passed to Gemini for coherent follow-ups.
- **Workflow integration:** Detects trigger phrases ("walk me through", "help me measure") and keyword patterns to start structured workflows.

4.2 Voice Pipeline — `genai.ts` (393 lines)

Two parallel pipelines:

English Pipeline

1. **STT:** `geminiTranscribeAudio()` — sends base64 audio blob to Gemini 2.5 Flash with prompt "Transcribe this audio exactly."
2. **LLM:** `askGemini()` — multi-turn contents array with system instruction defining Rhoda's persona. Accepts RAG context, instrument state, conversation history, and optional workflow context.
3. **TTS:** `geminiTTS()` — Gemini TTS preview model with Vindematrix voice. Returns raw PCM, converted to WAV via `pcmToWav()` helper (44-byte WAV header construction).

Twi Pipeline

1. **STT:** GhanaNLP ASR v1 — binary audio POST with `ocp-apim-subscription-key` header.
2. **Translation:** GhanaNLP v1 translate (tw → en) for processing, then (en → tw) for response.
3. **LLM:** Same Gemini call on English text.
4. **TTS:** GhanaNLP TTS v1 with `twi_speaker_7`.

4.3 Hand Ray & XR Interaction — `handRay.ts` (196 lines)

Renders visible pointer beams from XR hand/controller inputs using `CylinderGeometry` meshes (tapered from 2mm to 0.5mm, 3m length). Uses `targetRaySpace` for accurate pointer direction. Features:

- Per-material color: cyan (default), green (dashboard hit), yellow (avatar hit)
- Hit dot: 8mm sphere rendered at intersection point (`renderOrder: 1000` , `depthTest: false`)
- Raycasting against dashboard anchor and avatar mesh with recursive child intersection
- `handleSelect()` : UV-to-DOM coordinate mapping for virtual button clicks on PanelUI

4.4 Dashboard System — `dashboard.ts` (195 lines)

ECS system subscribing to PanelUI entities. Wires click handlers for 8 buttons: btn-talk, btn-lang, btn-identify, btn-summon, btn-follow, btn-reset, btn-allow, btn-deny. Refreshes status text every frame. Shows/hides confirmation buttons when a pending WRITE operation exists.

4.5 Text Chat Interface — `chat.ts` (304 lines)

Standalone entry point at `/chat.html`. Features:

- Message display with role-based styling (user, assistant, system)
- Sidebar: workflow launcher buttons, live instrument state panel, progress bars, transcript export
- Confirmation bar: appears when WRITE operations need approval
- Workflow step advancement via "next step" / "continue" keywords
- Anomaly detection routing for troubleshooting queries
- Full multi-turn conversation history (40 messages max)

4.6 RAG Retrieval — `rag.ts` (363 lines)

Client-side keyword-based retrieval over an embedded corpus of 14 documents covering procedures (6), safety rules (4), and fault patterns (4). Scoring: keyword match (+3), title match (+2), content frequency (up to +3). Returns top-K chunks with source attribution and category tag.

4.7 Instrument Mock & Bridge — `instrument.ts` (419 lines)

Dual-mode operation:

- **Local mock:** In-memory oscilloscope state (4 channels, horizontal, trigger, measurements). 10 function catalogue entries (3 READ, 7 WRITE). WRITE operations require `confirmed: true` or return `confirmation_required` with a human-readable prompt.
- **Flask bridge:** On startup, probes `/api/health` with 1.5s timeout. If reachable, routes all calls to Flask backend. Falls back to local mock on network error.
- **Safety limits:** Validated per-parameter (e.g., `scale_v_div: 0.001-100`, `level_v: -100-100`).

4.8 Anomaly Detection — `anomaly.ts` (237 lines)

Two-tier diagnosis:

1. **Rule-based:** 8 fault patterns (clipped, noisy, unstable, flat line, overvoltage, aliasing, ringing, DC offset) with keyword matching, confidence scores (0.70–0.95), fix step arrays, and unsafe flags.
2. **Gemini fallback:** For unrecognized patterns, sends description to Gemini with structured JSON output prompt. Parses `probable_cause`, `confidence`, `fix_steps`, and `unsafe_flag`.

Safety: The overvoltage pattern has confidence 0.95 and `unsafe: true`. The first fix step is always "IMMEDIATELY disconnect the probe from the circuit."

4.9 Progress Tracking — `progress.ts` (201 lines)

Rule-based competency scoring across 9 topic areas: `safety`, `probe_compensation`, `vertical_scale`, `horizontal_timebase`, `triggering`, `measurements`, `signal_identification`, `troubleshooting`, `component_identification`. Scoring formula:

- User engagement: `min(userTurns × 15, 40)`
- Assistant responses: `min(assistantTurns × 10, 30)`
- Tool calls: `min(toolCalls × 10, 20)`
- Errors: `-5 per error`
- Confirmations: `+5 per confirmed WRITE`

Curriculum ordering determines the recommended next topic. `getNudge()` returns a suggestion when the student has been on one topic for 5+ turns with score ≥ 60 .

4.10 Vision / Component ID — `vision.ts` (249 lines)

Two modes:

- **One-shot:** `captureFrame()` → `identifyComponent()`. Captures rear camera frame, sends base64 JPEG to Gemini with structured prompt. Returns label, description, confidence, and `safety_note`.
- **Live stream:** Opens Gemini Live API session. Streams camera frames at ~1fps via `sendRealtimeInput()`. Receives real-time narration of lab setup observations.

4.11 Guided Workflows — `workflows.ts` (189 lines)

Two workflow definitions:

Workflow	Category	Steps	Instrument Actions
Measure 1 kHz Sine Wave	Simple	10	7 (probe atten, enable CH1, V/div, timebase, trigger ×2, measurement)
Handle Overvoltage Warning	Safety-Critical	6	2 (probe atten, V/div) + 4 safety checks

Each step contains: instruction text, detailed explanation, optional `instrumentAction` (function name + params), and optional `safetyCheck` warning. `getWorkflowContext()` generates a prompt injection that tells Gemini exactly which step the student is on.

4.12 Session Logging — `session.ts` (168 lines)

Append-only log with schema: `{session_id, turn_id, role, content, tool_call?, confirmed?, topic?, timestamp}`. Persisted to localStorage and synced to Flask backend via POST. Supports export as JSON, session reset, and summary generation (turn count, topics covered, tool calls, confirmations).

5. Backend Implementation

5.1 Flask API Server — `app.py` (252 lines)

Unified backend combining instrument control, RAG retrieval, session logging, progress tracking, and failure point analytics. CORS-enabled for cross-origin frontend requests. Starts on port 5001 (configurable via `PORT` env var).

5.2 Mock Backend — `backends.py:MockScopeBackend`

Simulates an R&S RTB24 with 2 channels, trigger, and measurement state. Supports scoped reads (`all`, `acquisition`, `channel`, `trigger`) and path-based parameter writes (`channel.1.vertical_scale`, `trigger.level`, etc.). Measurement results return realistic default values (1000 Hz, 2.0 Vpp, 0.707 Vrms).

5.3 RTB24 Hardware Backend — `backends.py:RsInstrumentBackend`

Communicates with a real Rohde & Schwarz oscilloscope via `RsInstrument` (VISA/SCPI). Key features:

- **Auto-discovery:** `RsInstrument.list_resources("?*")` finds all connected instruments. `SCOPE_RESOURCE=USB` auto-selects the first USB device.
- **Path-to-SCPI mapping:** Internal map translates dot-notation paths (`channel.1.vertical_scale`) to SCPI commands (`CHANnel1:SCALE`).
- **Safe queries:** All SCPI reads wrapped in try/except with defaults — a failed query never crashes the server.
- **Measurement configuration:** Automatically sets measurement type (`MEASurement1:MAIN FREquency`), enables measurement (`MEASurement1:STATE ON`), then reads result (`MEASurement1:RESult:ACTual?`).

5.4 Confirmation Gate — `confirmation.py` (56 lines)

Every WRITE operation generates a unique `confirmation_id` (UUID hex) and stores a `PendingChange` with action, path, previous value, proposed value, and human-readable summary. Confirmation requires matching the ID, action, and path. Stale or mismatched tokens are rejected.

5.5 Server-Side RAG — `rag.py` (97 lines)

Loads 10 corpus documents from `server/rag/corpus/{procedure,safety,fault}/*.md`. Each document has YAML frontmatter (`source` , `category`) and markdown body. Embeds all documents at startup using `all-MiniLM-L6-v2` (sentence-transformers). Retrieval uses cosine similarity on normalized embeddings with optional category filtering.

6. SCPI Command Reference

Path	SCPI Command	Type	Description
<code>channel.N.enabled</code>	<code>CHANnelN:STATe {ON OFF}</code>	WRITE	Enable/disable channel
<code>channel.N.vertical_scale</code>	<code>CHANnelN:SCALE {value}</code>	WRITE	V/div setting
<code>channel.N.offset</code>	<code>CHANnelN:OFFSet {value}</code>	WRITE	Vertical offset
<code>channel.N.coupling</code>	<code>CHANnelN:COUPling {AC DC}</code>	WRITE	Input coupling mode
<code>trigger.source</code>	<code>TRIGger:A:SOURce {CH1 CH2 ...}</code>	WRITE	Trigger source channel
<code>trigger.level</code>	<code>TRIGger:A:LEVel1:VALue {value}</code>	WRITE	Trigger threshold (V)
<code>trigger.edge</code>	<code>TRIGger:A:EDGE:SLOPe {POS NEG}</code>	WRITE	Trigger edge direction
<code>trigger.mode</code>	<code>TRIGger:A:MODE {AUTO NORM SING}</code>	WRITE	Trigger mode
<code>timebase.scale</code>	<code>TIMEbase:SCALE {value}</code>	WRITE	Horizontal time/div
<code>acquisition.state</code>	<code>RUN / STOP</code>	WRITE	Start/stop acquisition
(identity)	<code>*IDN?</code>	READ	Instrument identification
(measurement)	<code>MEASurement1:RESult:ACTual?</code>	READ	Current measurement value

7. API Endpoint Specification

Method	Endpoint	Category	Request Body	Response
GET	<code>/api/health</code>	Health	—	<code>{status, instrument, rag, session_logs}</code>

GET	/api/ instrument/ state	READ	?scope=all	{ok, name, category, result}
POST	/api/ instrument/ set	WRITE	{path, value, confirmed, confirmationId}	{ok, needsConfirmation, confirmationId, summary}
POST	/api/ instrument/ measure	MEAS	{measurementType, source, confirmed}	{ok, result: {value, unit, status}}
POST	/api/ instrument/ confirm	WRITE	{path, value, confirmationId}	{ok, result}
GET	/api/ instrument/ discover	READ	—	{resources: [...], count}
POST	/api/rag/ retrieve	READ	{query, top_k, category}	{chunks: [{text, source, score}]}
POST	/api/session/ log	WRITE	{session_id, role, content, ...}	{ack, count}
GET	/api/session/ logs	READ	?session_id=	{logs: [...], count}
GET	/api/session/ transcript	READ	?session_id=	text/plain download
GET	/api/session/ progress	READ	?session_id=	{topic_scores, recommended_next}
GET	/api/session/ failure-points	READ	—	{total_errors, common_faults, ...}

8. Data Contracts

8.1 LogEvent

```
{
  session_id: string,      // e.g. "session_m1abc_x9z2"
  turn_id: number,        // monotonically increasing per session
  role: "user" | "assistant" | "system" | "tool",
  content: string,
```

```

tool_call?: {
  name: string,          // e.g. "set_timebase"
  params: Record<string, unknown>,
  category: "READ" | "WRITE",
  result?: unknown
},
confirmed?: boolean,
topic?: string,          // e.g. "triggering"
timestamp: string        // ISO 8601
}

```

8.2 CallFunctionResult

```

{
  status: "ok" | "error" | "confirmation_required",
  result?: unknown,
  error?: string,
  confirmation_prompt?: string // human-readable action description
}

```

8.3 DiagnoseResult

```

{
  probable_cause: string,    // "Signal clipping – input amplitude exceeds..."
  confidence: number,       // 0.0 - 1.0
  fix_steps: string[],      // ordered remediation steps
  unsafe_flag: boolean      // true if equipment/personal safety risk
}

```

8.4 WorkflowStep

```

{
  id: number,
  instruction: string,       // "Set probe attenuation to 10X."
  detail: string,           // extended explanation
  instrumentAction?: {      // auto-populated instrument call
    name: string,
    params: Record<string, unknown>
  },
  safetyCheck?: string      // safety warning if applicable
}

```

9. Demo Workflows

9.1 Workflow 1: Measure a 1 kHz Sine Wave (Simple)

Step	Instruction	Instrument Action
1	Connect probe to CH1 BNC input	—
2	Set probe attenuation to 10X	<code>set_probe_attenuation(ch=1, att="10X")</code>
3	Enable CH1	<code>set_channel_enabled(ch=1, enabled=true)</code>
4	Set vertical scale to 1 V/div	<code>set_vertical_scale(ch=1, scale=1.0)</code>
5	Set timebase to 500 μ s/div	<code>set_timebase(timebase=0.0005)</code>
6	Set trigger to CH1, rising edge, auto	<code>set_trigger(src=1, slope="rising", mode="auto")</code>
7	Adjust trigger level to midpoint	<code>set_trigger(src=1, level=1.0)</code>
8	Verify waveform is stable	—
9	Read measurement values	<code>get_measurement()</code>
10	Review results	—

9.2 Workflow 2: Handle Overvoltage Warning (Safety-Critical)

Step	Instruction	Safety Check
1	STOP — Disconnect probe immediately	 Overvoltage can damage instrument and create safety hazard
2	Check signal voltage with multimeter	 Never assume voltage is safe — always verify
3	Verify voltage within safe limits	 Exceeding limits can permanently damage input amplifier
4	Select correct probe attenuation	Instrument: <code>set_probe_attenuation(ch=1, att="10X")</code>
5	Set oscilloscope to maximum V/div	Instrument: <code>set_vertical_scale(ch=1, scale=10.0)</code>
6	Reconnect probe carefully and verify	—

10. Testing

10.1 Backend API Tests (11 tests, all passing)

Test	What It Validates
test_health	Health endpoint returns status, instrument mode, RAG status
test_instrument_state_read	READ state returns ok=true with result object
test_instrument_set_requires_confirmation	WRITE without confirmed=true returns needsConfirmation
test_instrument_set_confirmed	WRITE with valid confirmationId succeeds
test_instrument_measure	Measurement endpoint returns result
test_instrument_discover	Discovery endpoint returns resources array
test_session_log_and_retrieve	Log event, then retrieve by session_id
test_session_transcript_export	Transcript exports as text/plain with correct content
test_session_progress	Progress endpoint returns topic_scores and recommended_next
test_failure_points	Failure points endpoint returns error/fault aggregation
test_rag_retrieve	RAG retrieval returns chunks array

```
$ python server/test_api.py
PASS test_failure_points
PASS test_health
PASS test_instrument_discover
PASS test_instrument_measure
PASS test_instrument_set_confirmed
PASS test_instrument_set_requires_confirmation
PASS test_instrument_state_read
PASS test_rag_retrieve
PASS test_session_log_and_retrieve
PASS test_session_progress
PASS test_session_transcript_export
```

```
11 passed, 0 failed out of 11
```

11. Completion Assessment

11.1 Track 2 Deliverables

Deliverable	Status	Evidence
Live demo: ≥2 workflows (simple + safety-critical)	DONE	workflows.ts: "Measure 1kHz Sine" (10 steps) + "Handle Overvoltage" (6 steps)
Assistant guides user through workflow	DONE	Step-by-step with Gemini context injection via <code>getWorkflowContext()</code>
Instrument configuration after confirmation	DONE	instrument.ts confirmation gate + dashboard btn-allow/ btn-deny
Conversation transcripts	DONE	session.ts <code>exportJSON()</code> + server <code>/api/session/transcript</code>
Code repo	DONE	github.com/PhidLarkson/rohde-schwarz-assistant
README on connecting to instruments/simulator	DONE	README.md: Simulator Mode + RTB24 via USB/LAN sections

11.2 Evaluation Criteria

Metric	Status	Implementation
Helpfulness: user task success rate	DONE	Guided workflows with step tracking, RAG-grounded responses
Accuracy: correctness of instructions	DONE	RAG corpus from R&S documentation, safety limits enforced
Accuracy: safety checks	DONE	Confirmation gate, overvoltage detection, safety corpus
Robustness: ambiguous queries	PARTIAL	Gemini handles ambiguity; no explicit disambiguation prompts
Robustness: error recovery	DONE	Pipeline error handler with status display and auto-retry

11.3 Stretch Goals

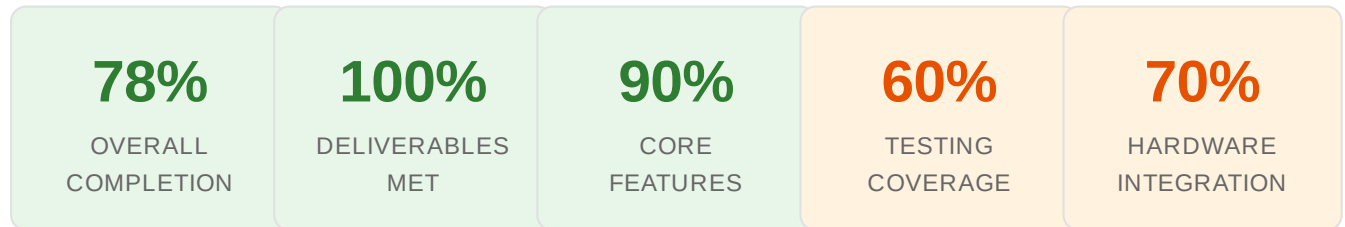
Goal	Status	Notes
Voice integration	DONE	Full duplex: Gemini STT → LLM → TTS with lip sync
Auto-configure instruments after confirmation	DONE	Real RTB24 SCPI commands via <code>RsInstrumentBackend</code>

Adapt explanations to skill level

PARTIAL

Progress tracking identifies weak areas; nudge system suggests topics

11.4 Overall Completion Scale



Remaining 22% consists of: end-to-end testing with real hardware (8%), explicit disambiguation prompting for ambiguous queries (4%), latency measurement instrumentation (3%), privacy consent flow (3%), more corpus documents (2%), production error handling hardening (2%).

12. Oscilloscope Testing Procedure

12.1 Prerequisites

1. R&S RTB24 oscilloscope connected via USB
2. R&S VISA driver installed (from rohde-schwarz.com)
3. Python package: `pip install RsInstrument`
4. Backend server running: `python server/app.py`

12.2 Step-by-Step Testing

1

Verify USB Connection

```
python3 -c "from RsInstrument import RsInstrument;
print(RsInstrument.list_resources('*'))"
```

Expected output: a list containing your instrument's VISA address, e.g. `['USB0::0x0AAD::0x01D6::102345::INSTR']`. If empty, check USB cable and VISA driver installation.

2 Start Backend in Hardware Mode

```
export SCOPE_RESOURCE=USB
python server/app.py
```

The server should print: `Instrument: hardware (USB0:...)`. If it says "mock mode", the auto-discovery failed.

3 Test READ — Instrument State

```
curl http://localhost:5001/api/instrument/state?scope=all | python3 -m json.tool
```

Verify: `ok: true`, `instrument_id` matches your RTB24 IDN string, channel states reflect physical scope settings.

4 Test WRITE — Set Timebase (with confirmation)

```
# Step 1: Preview the change
curl -X POST http://localhost:5001/api/instrument/set \
  -H "Content-Type: application/json" \
  -d '{"path":"timebase.scale","value":0.001,"confirmed":false}'

# Note the confirmationId from the response

# Step 2: Confirm the change
curl -X POST http://localhost:5001/api/instrument/confirm \
  -H "Content-Type: application/json" \
  -d '{"path":"timebase.scale","value":0.001,"confirmationId":"YOUR_ID_HERE}"'
```

Verify: The oscilloscope's timebase should physically change to 1 ms/div. Check the scope display.

5 Test WRITE — Set Vertical Scale

```
# Preview
curl -X POST http://localhost:5001/api/instrument/set \
  -H "Content-Type: application/json" \
  -d '{"path":"channel.1.vertical_scale","value":0.5,"confirmed":false}'

# Confirm with the returned confirmationId
curl -X POST http://localhost:5001/api/instrument/confirm \
  -H "Content-Type: application/json" \
  -d '{"path":"channel.1.vertical_scale","value":0.5,"confirmationId":"YOUR_ID"}'
```

Verify: CH1 vertical scale changes to 500 mV/div on the scope.

6 Test Measurement

Connect a signal source (e.g., probe comp output) to CH1.

```
curl -X POST http://localhost:5001/api/instrument/measure \
  -H "Content-Type: application/json" \
  -d
'{"measurementType":"frequency","source":"CH1","confirmed":true,"confirmationId":"skip"}'
```

Verify: The returned `value` should match the physical signal frequency (e.g., ~1000 Hz for the probe comp output).

7 Test via Chat Interface

```
# Start frontend
npx vite
```

Open `https://localhost:8081/chat.html`. Click "Measure 1 kHz Sine" workflow. Walk through each step — each instrument-changing step should show a confirmation dialog. Approve each step and verify the oscilloscope physically changes settings.

8 Test Free Conversation with Real Scope

In the chat, type: "What is the current oscilloscope state?" — Rhoda should respond with real values from the instrument. Then type: "Set the timebase to 2ms per division" — Rhoda should ask for confirmation, and on approval, the physical scope should change.

9

Verify Safety Gate

Type: "Set the vertical scale to 200 V/div". The system should reject this because `scale_v_div` has a safety limit of max 100 V/div. Verify the error message is returned and the scope is NOT changed.

10

Export Transcript

Click "Export Transcript" in the chat sidebar. Verify the JSON file contains all conversation turns, tool calls, and confirmation events with timestamps.

Safety Warning: When testing with real hardware, always start with the oscilloscope in a known safe state. Use the probe compensation output as a safe signal source. Never connect to high-voltage circuits during testing. Keep `SCOPE_RESOURCE` unset (mock mode) for all development and debugging.

13. Safety Considerations

- **Confirmation Gate:** Every WRITE instrument command requires explicit user confirmation via a confirmation token. No silent writes.
- **Safety Limits:** Parameter ranges are enforced before execution (e.g., vertical scale 0.001–100 V/div, trigger level ± 100 V).
- **Overvoltage Detection:** Keyword patterns trigger immediate disconnect guidance with `unsafe_flag: true`.
- **Default to Simulator:** The system runs in mock mode unless `SCOPE_RESOURCE` is explicitly set.
- **Probe Safety:** RAG corpus includes probe inspection, grounding, and voltage limit documentation.
- **No PII Logging:** Session logs store session IDs and content only. No user names, emails, or identifiers.
- **Read-Only by Default:** READ operations never require confirmation and cannot alter instrument state.

14. Known Limitations & Future Work

Current Limitations

- **No explicit disambiguation:** Ambiguous queries are sent directly to Gemini without asking clarifying questions.
- **No latency instrumentation:** Response times are not measured or reported (spec target: <2s for local queries).

- **Session persistence:** Server-side sessions are in-memory. A server restart loses all logs (localStorage persists on client).
- **SCPI coverage:** The RTB24 backend covers the most common commands but not the full SCPI command set.
- **Privacy consent:** No explicit consent prompt for logging (spec requires "do not log PII without consent").
- **XR dashboard interaction:** UV-to-DOM coordinate mapping depends on PanelUI internal rendering; may not work on all headsets.

Recommended Next Steps

1. Add end-to-end testing with real RTB24 hardware
2. Add response latency timing and dashboard metric
3. Implement explicit disambiguation prompts for ambiguous queries
4. Add server-side session persistence (SQLite or file-based)
5. Expand SCPI command coverage (waveform capture, math channels)
6. Add privacy consent banner at session start
7. Integrate adaptive difficulty: adjust explanation depth based on progress score